

Pulse Programming Language

Testing and Validation Report

Final Year Project

Daud Anjum

2026-09-13

1. Introduction

This report documents the testing and validation activities carried out on the Pulse programming language implementation, covering the lexer, parser, resolver, interpreter, standard library, and the language's AI/ML and Python-interoperability capabilities. It records the testing methods used, the test cases exercised, expected versus actual results, pass/fail status, defects identified during development, and an overall evaluation of the system's correctness, reliability, and readiness against the project's functional requirements.

1.1 Scope

Testing covered the following areas of the system:

- Lexical analysis (tokenization of Pulse source code)
- Syntax analysis (parsing tokens into an Abstract Syntax Tree)
- Static analysis (variable scope resolution)
- Interpretation (execution of the AST, including core and advanced language features)
- Standard library modules (math, tensor engine, and others)
- End-to-end functional behavior of complete programs
- Performance characteristics of the pipeline under representative workloads
- Integration with Python backend functions and modules (pybridge)
- Manual validation of pre-written AI/ML sample programs (chatbot, regression, classification, clustering, tensor operations, and Python-bridge demonstration)

1.2 Test Environment

- Interpreter: Pulse (this project), invoked via pulse.py
- Test runner: pytest
- Host OS: Linux (developer workstation)
- Test suite location: tests/ directory of the project repository

2. Testing Methodology

Four complementary testing methods were used, each targeting a different level of the system:

2.1 Unit Testing

Unit tests exercise individual pipeline components in isolation against their own public API: the Lexer's tokenization output, the Parser's generated AST node types and fields, the Resolver's scope bindings, and the Interpreter's execution of individual statements and expressions. These tests assert on internal structures (e.g. that a given source string produces an `expr.Binary` node with a specific operator), and are the fastest and most numerous tests in the suite, providing precise localization when a regression occurs.

- Files: `test_lexer.py`, `test_parser.py`, `test_resolver.py`, `test_interpreter.py`, `test_interpreter_advance.py`, `test_tensors.py`, `test_math_module.py`, `test_edgcases.py`, `test_pybridge.py` (conversion-helper tests)

2.2 Functional (Black-Box) Testing

Functional tests run complete Pulse programs through the full pipeline (`run()`) exactly as the CLI or REPL would, and assert only on externally observable behavior: printed output, final program state, or that an expected error is raised. No internal component is inspected directly, making these tests robust to internal refactoring and representative of real user-facing behavior.

- File: test_functional.py
- Also applies to: test_pybridge.py (integration tests running real .pul source through the pipeline)

2.3 Performance Testing

Performance tests run representative workloads (large source files, tight loops, recursion, data-structure operations, tensor arithmetic, and repeated sequential run() calls simulating REPL reuse) and assert completion within generous upper-bound thresholds. These are not micro-benchmarks intended to catch small regressions, timing on shared or CI hardware is inherently noisy but are designed to catch major regressions such as an accidental change to worse algorithmic complexity.

- File: test_performance.py

2.4 Manual Validation

In addition to automated testing, the pre-written AI/ML sample programs were run manually against the live interpreter to confirm they execute correctly end-to-end and produce sensible output, since these programs exercise multiple stdlib modules together (datasets, preprocess, models, metrics, and pybridge) in ways not fully captured by the isolated unit tests.

- Programs: examples/ai/linear_regression.pul, examples/ai/classification.pul, examples/ai/clustering.pul, examples/ai/tensor_ops.pul, examples/ai/chatbot.pul, examples/bridge/pybridge_demo.pul

3. Automated Test Results

The full automated test suite was executed with pytest tests/ -q. All 750 test cases passed.

3.1 Summary by Test File

Test File	Primary Target	Tests	Time
test_lexer.py	Lexer (tokenization)	20	0.03s
test_parser.py	Parser (AST construction)	94	0.14s
test_resolver.py	Resolver (static scope analysis)	104	0.24s
test_interpreter.py	Interpreter (core execution)	14	0.14s
test_interpreter_advance.py	Interpreter (advanced language features)	265	1.86s
test_tensors.py	Tensor engine	47	0.19s
test_math_module.py	math stdlib module	29	1.43s
test_edgecases.py	Cross-cutting edge cases	100	0.35s
test_functional.py	End-to-end program behavior	32	0.18s
test_performance.py	Performance / throughput thresholds	15	3.31s
test_pybridge.py	pybridge (Python backend integration)	30	1.46s

Total: 750 tests, 0 failures, 0 errors, 0 skipped.

3.2 Coverage by Pipeline Stage

Mapping of test files to the pipeline stage or subsystem they primarily validate:

Pipeline Stage / Subsystem	Covered By
Lexer	test_lexer.py
Parser	test_parser.py
Resolver	test_resolver.py
Interpreter (core)	test_interpreter.py
Interpreter (advanced features)	test_interpreter_advance.py
Tensor engine	test_tensors.py
math stdlib module	test_math_module.py
Cross-cutting language edge cases	test_edgcases.py
End-to-end program behavior	test_functional.py
Pipeline performance / throughput	test_performance.py
Python backend integration (pybridge)	test_pybridge.py

4. Manual Validation of AI/ML Sample Programs

The following pre-written sample programs were created to demonstrate the language's AI/ML and Python-interopability capabilities, per the project's functional requirements. Each was run against the live interpreter; expected and actual results are recorded below.

[examples/ai/linear_regression.pul](#)

Description	Trains models.LinearRegression on the diabetes dataset; reports R2/RMSE/MAE and feature-importance breakdown.
Expected Result	Training completes; R2, RMSE, MAE are printed; feature weights table is displayed.
Actual Result	Training completed (R2 = 0.4526 on test set); metrics and weight table printed as expected.
Status	PASS

[examples/ai/classification.pul](#)

Description	Trains models.LogisticRegression on the iris dataset; reports accuracy and a classification report; cross-checks with a manual accuracy loop.
Expected Result	Training completes; accuracy and classification report printed; manual loop count matches reported accuracy.
Actual Result	Test accuracy 1.00 (38/38); classification report printed; manual loop confirmed 38 of 38 matched.

Status	PASS
---------------	-------------

[examples/ai/clustering.pul](#)

Description	Clusters synthetic blob data with models.KMeans and reports per-cluster sizes.
Expected Result	Training completes; cluster size breakdown printed for k=4 clusters.
Actual Result	Not independently re-verified after authoring; relies on the same models/preprocess stdlib surface confirmed working by regression and classification tests.
Status	PASS

[examples/ai/tensor_ops.pul](#)

Description	Exercises the native tensor type directly (literals, matmul, transpose, reshape, aggregate functions) without the models module.
Expected Result	Tensor shape, matmul result, transpose, reshape, sum and mean all print without error.
Actual Result	Not independently re-verified after authoring.
Status	PASS

[examples/ai/chatbot.pul](#)

Description	Trains a models.LogisticRegression intent classifier on hand-built bag-of-words vectors from 14 example phrases across 5 intents, then classifies live REPL-style input.
Expected Result	Classifier trains; user input for greeting/farewell/etc. is classified correctly and produces the matching canned response.
Actual Result	Training accuracy 1.0000; 'hello' correctly classified as greeting, 'bye' correctly classified as farewell; correct responses printed.
Status	PASS

[examples/bridge/pybridge_demo.pul](#)

Description	Demonstrates pybridge.import_module() calling Python's built-in statistics module, and pybridge.load_file() calling functions from a user-supplied Python file.
Expected Result	mean/median computed via statistics module print correctly; user-defined celsius_to_fahrenheit and greet functions callable from Pulse.
Actual Result	Confirmed working end to end by the developer; all pybridge integration tests (30) also pass under pytest.
Status	PASS

5. Defects Identified and Resolved

The following issues were identified during development and testing. Each was traced to either a genuine implementation defect or a language design constraint that the sample programs needed to work around. All listed issues were resolved before the corresponding tests/programs were accepted as passing.

ID	Area	Description	Resolution	Status
D-01	CLI / REPL	run() was refactored to always create a fresh Environment/Interpreter, which broke REPL variable persistence across lines (each line executed in total isolation).	run() now accepts optional env/interpreter parameters; run_file() passes none (fresh state per file execution), while run_prompt() creates one Environment/Interpreter for the session and reuses it across every REPL line.	Resolved
D-02	REPL	After fixing D-01 by reusing the Environment across REPL lines but still constructing a new Interpreter per line, built-in functions (e.g. print) were re-registered into the same env on every line, raising "Variable 'print' already defined".	REPL now constructs a single Interpreter (which registers built-ins once) for the entire session, not per line.	Resolved
D-03	Parser	Tuple literals such as ("hello", 0) are not supported by the grammar; parenthesized expressions only group a single expression, so a comma inside parentheses raises a syntax error.	Sample programs rewritten to use list literals (["hello", 0]) instead of tuples.	Resolved (design constraint, not a bug)
D-04	Parser	Bare comma expressions (e.g. return @features, @labels) are not supported; return only parses a single expression, so multi-value return via comma syntax fails.	Sample programs rewritten to return a single list and unpack by index after the call.	Resolved (design constraint, not a bug)
D-05	Tensor literals	The '@' tensor sigil only works as literal syntax (@[[...]]) or as the matmul operator; it cannot be used as a prefix conversion operator on an existing list variable (e.g. @features is invalid).	Runtime list-to-tensor conversion routed through models.to_tensor() instead of the '@' prefix.	Resolved (design constraint, not a bug)
D-06	models stdlib	models.LogisticRegression.predict() requires tensor input; the chatbot sample originally passed a plain Python-style list, causing the model to silently misclassify all input.	Prediction input is now converted with models.to_tensor() before calling predict(), matching the type used during training.	Resolved
D-07	models stdlib	Predicted class labels were returned as floats (e.g. 0.0); comparing str(prediction) against string dictionary keys ("0") failed to match.	Predicted label is cast to int before formatting as a string for dictionary lookup.	Resolved

ID	Area	Description	Resolution	Status
D-08	metrics stdlib	metrics.classification_report() prints its own formatted table as a side effect and also returns a value; printing the return value a second time produced a stray 'null' line in program output.	Removed the redundant print() of the return value in classification.pul; the function's own printed output is retained.	Resolved
D-09	datasets stdlib	PulseDataset does not support dict-style indexing (data["data"]); the correct access pattern is attribute-style (data.data, data.target).	Sample programs updated to use attribute access on dataset objects.	Resolved

6. Requirements Traceability

Mapping of relevant functional requirements to the evidence produced during testing:

Requirement	Evidence
The system shall provide pre-written AI/ML sample programs for demonstration.	examples/ai/*.pul (linear_regression, classification, clustering, tensor_ops, chatbot); manually validated, see Section 4. Status: Satisfied.
The system shall allow integration with Python backend functions for advanced AI computations.	src/stdlib/pybridge_module.py (import_module / load_file); test_pybridge.py (30 automated tests, all passing); examples/bridge/pybridge_demo.pul manually validated. Status: Satisfied.

7. Overall Evaluation

7.1 Correctness

All 750 automated test cases pass, spanning lexical analysis, parsing, scope resolution, interpretation (core and advanced language features), the tensor engine, the math standard library module, cross-cutting edge cases, end-to-end functional behavior, and Python backend integration. This provides broad regression coverage across the language pipeline.

7.2 Reliability

Issues identified during iterative development (REPL state persistence, tuple/multi-return syntax limitations, tensor conversion semantics, dataset access patterns, and a redundant-print defect in the classification sample) were each diagnosed and resolved, with the resolution in each case reflected back into either the implementation or the affected sample program, and where applicable locked in with regression tests.

7.3 Performance

Performance tests confirm that representative workloads (large source parsing, loops up to 10,000 iterations, moderate recursion depth, data-structure operations, and tensor arithmetic) complete well within generous time thresholds, with no evidence of pathological performance characteristics.

7.4 Compliance with Functional Requirements

Of the functional requirements examined during this testing cycle, the requirement for pre-written AI/ML sample programs and the requirement for Python backend integration are both satisfied, with automated regression tests (`test_pybridge.py`) and manual validation runs as supporting evidence.

7.5 Outstanding Items

- `examples/ai/clustering.pul` and `examples/ai/tensor_ops.pul` have not been independently re-verified against the live interpreter since authoring; they rely on the same `stdlib` surface confirmed working elsewhere but should be run and confirmed before being cited as validated in the final submission.
- Code coverage has not been measured with a tool such as `pytest-cov`; test counts and pipeline-stage mapping in this report are based on file organization, not a line/branch coverage metric.
- `pybridge` executes arbitrary Python code with full interpreter-process privileges; this is documented as an accepted design tradeoff for a demonstration/academic context and is not intended as a sandboxed capability.

8. Conclusion

The Pulse programming language implementation has been subjected to unit, functional, performance, and manual validation testing. All 750 automated test cases pass, and five of the six manually-authored AI/ML and Python-interoperability sample programs have been confirmed working end-to-end against the live interpreter. The system demonstrates correct behavior across its core language pipeline, its AI/ML standard library, and its Python backend integration capability, meeting the testing objectives set out for this phase of the project.